FLORIDA UNIVERSITY SOUTHEAST

FUSE AI-201

AI Model Development and Implementation

Group 5

Project Proposal

Title:

Real-Time Network Intrusion Detection Using Multi-Class Machine Learning Classification

Assigned Domain: Cyber security

Group Members:

1. Wogari Ararsa
2. Amanuel Ayalew Arega
3. Eyob Daniel Asefa
4. Surafiel Habib Asefa
5. Mulatie Yeshu Asegie
6. Asamn Admasu Asfaw
7. PETROS TARIKU ASSEFA
8. Semegn Mulugeta Assefa
9. Fuad KedirAssen
10. Mohammed Yassin Awel
11. Tenaw Workie Awoke

Oct 6/20026

Addis Ababa

Table Content

Table Content 1

1. Problem Statement and Domain Context 3

1.1. Domain Context 3

1.2. Problem Statement 3

1.3. Current Approach and Limitations 3

1.4. Problem Consequences 4

1.5. Evidence and References 4

2. Target Users and Stakeholders 5

3. Dataset Identification and Acquisition Plan 6

3.1. Data Preparation Expectations 7

3.2. Data Feasibility 7

4. ML/DL Task Specification 8

5. Proposed AI/ML Approach 8

5.1. Primary Model/Model Family 8

5.2. Why This Model Is Appropriate 8

5.3. Baseline 9

5.4. Alternative/Fallback Model 9

5.5. Key Features/Inputs 9

5.6. Required Compute and Tools 10

5.7. Initial Implementation Considerations 10

6. Success Criteria and Evaluation Plan 10

6.1. Evaluation Design 10

6.2. Technical Success Criteria 11

6.3. Practical and Business Criteria 12

7. MVP and Project Scope 12

7.1. MVP: Smallest Useful Version 12

7.2. In Scope..... 12
7.3. Out of Scope..... 13
7.4. Future Extensions..... 13
8. Model Development Roadmap..... 14
9. Proposed Application Concept..... 15
10. Intended Deployment Direction..... 16
11. Resources and Group Responsibilities..... 17
11.1. Technical Resources..... 17
12. Project Timeline..... 18
13. Risk Assessment and Mitigation..... 19
14. Ethical, Privacy, and Responsible AI..... 20
15. Expected Project Outcomes..... 21
16. Future Improvements..... 22
17. Executive Summary..... 23
18. References..... **24**

1. Problem Statement and Domain Context

1.1. Domain Context

Cyber security focuses on protecting computer systems, networks, devices, applications, and data from unauthorized access, misuse, disruption, and attack. A key cyber security challenge is identifying malicious network activity before it causes significant damage.

Network intrusion detection involves monitoring network traffic and identifying activity that differs from expected or normal behavior. A network connection can contain information such as protocol type, duration, source and destination behavior, number of bytes transferred, error counts, login activity, and other statistical characteristics.

1.2. Problem Statement

Organizations need a practical way to classify network traffic as normal or as a specific attack category in near real time. The proposed project will develop a multi-class machine-learning classifier using the NSL-KDD dataset. The system will analyze network-traffic features and produce one of the following types of predictions:

- Normal traffic
- Denial-of-service attack
- Probe or scanning attack
- Remote-to-local attack
- User-to-root attack

The exact labels and grouping strategy will be confirmed during Module 5 after inspecting the dataset and defining the target encoding.

1.3. Current Approach and Limitations

Current intrusion-detection approaches commonly include:

1. **Signature-based detection:** Matches traffic against known attack signatures.
2. **Rule-based systems:** Uses manually written rules and thresholds.

3. **Statistical or anomaly-based detection:** Identifies activity that differs from an established baseline.
4. **Manual security analysis:** Security analysts investigate alerts and traffic records.

These approaches have limitations. Signature-based systems may not recognize new or modified attacks. Rule-based systems require continuous expert maintenance and may produce many false alerts. Anomaly detection can mistake legitimate unusual activity for an attack. Manual analysis is time-consuming and difficult to scale.

A machine-learning classifier could support security analysts by identifying patterns in historical traffic and producing consistent predictions quickly. It should be treated as a decision-support tool rather than a replacement for expert investigation.

1.4. Problem Consequences

If intrusions are not detected promptly, attackers may:

- Steal or modify sensitive information.
- Interrupt network services.
- Install malware or establish persistent access.
- Move laterally through the network.
- Increase recovery and investigation costs.
- Damage customer trust and organizational reputation.

High false-positive rates also create problems because analysts may become overwhelmed by alerts and ignore legitimate warnings.

1.5. Evidence and References

The proposal will be supported by credible cyber security and dataset sources, including:

- The NSL-KDD dataset documentation.
- Research on intrusion-detection systems and machine-learning classification.
- Guidance from recognized cyber security organizations such as NIST.
- Documentation for the selected machine-learning algorithms and evaluation metrics.

2. Target Users and Stakeholders

Stakeholder/User	Role or Relationship to Problem	Need/Expected Value	Decision or Action Supported
Security analysts	Investigate suspicious network activity	Faster prioritization of potentially malicious traffic	Decide which alerts require investigation
Network administrators	Maintain network availability and performance	Visibility into abnormal or disruptive traffic	Review network controls and isolate affected systems
Security operations center personnel	Monitor security events continuously	Consistent classification of incoming events	Escalate or prioritize security incidents
IT managers	Responsible for organizational technology risk	Summary information about intrusion patterns	Support resource allocation and security planning
Data scientists/developers	Build and evaluate the system	Structured data and measurable performance objectives	Select and improve the model
Organization or system owner	Affected by cyber incidents	Reduced operational and security risk	Approve use of the prototype or future pilot

The primary users of the proposed prototype will be security analysts and network administrators. They will use the model's output to prioritize investigation; the model will not independently authorize defensive actions.

3. Dataset Identification and Acquisition Plan

Dataset Item	Group Response/Evidence
Dataset/source name	NSL-KDD Dataset
Access URL/reference	Public benchmark dataset available through established academic and data-science repositories, including Kaggle and the Canadian Institute for Cybersecurity
Records/observations	Approximately 125,000 connection records across the training and testing files, depending on the selected files and preprocessing approach
Features/variables	Network-connection characteristics, including duration, protocol type, service, connection status, byte counts, error indicators, login-related features, host-level statistics, and other traffic attributes
Target/labels	Normal traffic and attack labels, which may be mapped into broader attack categories such as DoS, Probe, R2L, and U2R
Format	Typically CSV or text-based tabular files
Time/geographic coverage	Benchmark network-connection records; not a current live-traffic stream and not representative of a specific modern geographic region
Known data-quality issues	Categorical variables require encoding; class distributions are uneven; duplicate or highly similar observations may affect evaluation; some features may be correlated; the dataset may not reflect current attack techniques
Representativeness/bias limitations	NSL-KDD is a historical benchmark and may not represent modern enterprise networks, encrypted traffic, cloud environments, Internet-of-Things devices, or current attacker behavior

Access, legal, or privacy considerations	The dataset is publicly available for research and educational use. The project will verify the applicable license and citation requirements before use. It does not require collection of personally identifiable information
Fallback data source/plan	If the primary source is unavailable, use the original KDD Cup 1999 data or another public intrusion-detection dataset, such as CIC-IDS2017, subject to time and access constraints

3.1. Data Preparation Expectations

Before model development, the group will:

- Inspect column names, data types, and label definitions.
- Check for missing values, duplicates, invalid values, and extreme outliers.
- Encode categorical features.
- Scale numerical features where required by the selected algorithms.
- Analyze class distributions.
- Separate predictors from the target label.
- Avoid data leakage by fitting preprocessing steps only on the training data.
- Document all transformations so that the same process can be used by the application.

3.2. Data Feasibility

The dataset is feasible for the project because it is publicly accessible, tabular, manageable on ordinary computing hardware, and already widely used for intrusion-detection experiments. Its main limitation is that strong performance on NSL-KDD may not translate directly to live operational networks. This limitation will be clearly stated in the final documentation.

4. ML/DL Task Specification

Element	Specification
Business/industry problem	Detect and classify potentially malicious network traffic
ML/DL task type	Supervised multi-class classification
Input/features/information	Network-connection features available at prediction time
Target variable/label	Normal traffic or one of several attack categories
Prediction point/horizon	At the time a network connection or traffic record is analyzed
Model output	Predicted traffic class, class probabilities or confidence scores, and evaluation information
Who will use the output?	Security analysts, network administrators, and other authorized cybersecurity personnel

The system will not predict the future occurrence of an attack. Instead, it will classify a network-traffic record based on its observed features.

5. Proposed AI/ML Approach

5.1. Primary Model/Model Family

The primary candidate model will be a **Random Forest classifier**.

5.2. Why This Model Is Appropriate

Random Forest is suitable because:

- It performs well on structured tabular data.
- It can model nonlinear relationships between network features and attack classes.
- It is generally less sensitive to feature scaling than distance-based or gradient-based methods.
- It can provide feature-importance information.
- It is practical to train using ordinary CPU resources.

- It provides a strong, interpretable benchmark before considering more complex deep-learning methods.

A Gradient Boosting model or neural network may also be evaluated if time and resources permit. The final model will be selected based on validation results, class-specific performance, computational requirements, and suitability for application integration.

5.3. Baseline

The baseline will be one or both of the following:

- A majority-class classifier.
- A Logistic Regression or Decision Tree classifier.

The majority-class baseline will show how well a trivial method performs. Logistic Regression or Decision Tree will provide a meaningful, simple machine-learning comparison.

5.4. Alternative/Fallback Model

If Random Forest is unsuitable, the group will test:

- Logistic Regression with appropriate encoding and scaling.
- Decision Tree.
- Gradient Boosting.
- Multilayer Perceptron, if computationally feasible.

5.5. Key Features/Inputs

Important feature groups are expected to include:

- Protocol type.
- Network service.
- Connection status.
- Duration.
- Source and destination byte counts.
- Failed login indicators.
- Number of connections to the same host.
- Error and fragmentation indicators.

- Host-based and connection-based traffic statistics.

Feature importance and error analysis will be used to determine which features are most relevant.

5.6. Required Compute and Tools

Expected tools include:

- Python.
- Google Colab.

A GPU is not expected to be necessary for the MVP.

5.7. Initial Implementation Considerations

The preprocessing pipeline and model must be saved together or reproduced consistently so that application inputs receive the same transformations used during training. The application should validate inputs, handle unsupported values, display prediction confidence carefully, and avoid presenting predictions as guaranteed security conclusions.

6. Success Criteria and Evaluation Plan

6.1. Evaluation Design

The dataset will be divided into training, validation, and test sets using a stratified approach where possible. The test set will remain isolated until final evaluation. Cross-validation may be used on the training data for model comparison and hyperparameter selection.

Because attack categories may be imbalanced, accuracy alone will not be sufficient. The group will report:

- Macro-averaged precision.
- Macro-averaged recall.
- Macro-averaged F1-score.
- Weighted F1-score.
- Per-class precision, recall, and F1-score.
- Confusion matrix.
- Inference latency where measurable.

6.2. Technical Success Criteria

Metric/Criterion	Target	Why It Matters	How It Will Be Evaluated
Primary metric: macro F1-score	At least 0.80 on the held-out test set	Gives each attack class meaningful importance	Calculate macro F1 after final testing
Macro recall	At least 0.80	Measures the ability to detect all classes, including minority classes	Report class-wise and macro recall
Weighted F1-score	At least 0.85	Reflects overall performance while considering class frequency	Calculate on held-out test data
Normal-class precision	At least 0.90	Reduces unnecessary analyst alerts	Use the classification report and confusion matrix
Attack recall	At least 0.85 overall, subject to class feasibility	Missed attacks can create serious security risks	Analyze false negatives by class
Baseline comparison	Improve macro F1 by at least 10 percentage points over the selected baseline	Demonstrates value beyond a simple model	Compare identical test metrics
Confusion-matrix coverage	Identify and explain major class confusions	Shows where the system is weak	Review errors by predicted and true class
Inference performance	Prototype prediction returned within approximately one second for a single record	Supports interactive demonstration	Measure application response time locally

Targets are initial project goals and may be adjusted only if justified by data quality, class scarcity, or documented experimental results.

6.3. Practical and Business Criteria

The project will be considered practically successful if:

- A user can submit a valid traffic record through the prototype.
- The system returns a predicted class and confidence information.
- The result is understandable to a nontechnical demonstration audience.
- The application handles invalid or incomplete input safely.
- The model and preprocessing pipeline can be reproduced.
- The limitations of the dataset and prototype are clearly documented.

7. MVP and Project Scope

7.1. MVP: Smallest Useful Version

The MVP will be a prototype that:

1. Loads and preprocesses the NSL-KDD dataset.
2. Trains and evaluates at least one baseline and one stronger classifier.
3. Saves the selected preprocessing pipeline and model.
4. Accepts a network-traffic record through a simple interface.
5. Returns a predicted class and confidence or probability information.
6. Displays basic model-performance results.
7. Provides a short explanation that the output is decision support and requires analyst verification.

7.2. In Scope

- NSL-KDD data acquisition and preparation.
- Exploratory data analysis.
- Multi-class attack classification.
- Baseline and candidate-model comparison.

- Class-imbalance analysis.
- Evaluation using precision, recall, F1-score, and confusion matrices.
- A local or hosted demonstration interface.
- Basic input validation.
- Technical documentation and final presentation.

7.3. Out of Scope

- Automated blocking or quarantining of network connections.
- Direct integration with enterprise firewalls, SIEM systems, or routers.
- Continuous live packet capture.
- Production-grade security monitoring.
- Guaranteed detection of zero-day attacks.
- Collection of personally identifiable information.
- Full-scale cloud infrastructure.
- Mobile application development.
- Comprehensive adversarial-robustness testing.

7.4. Future Extensions

- Test on a modern intrusion-detection dataset.
- Add streaming or live-traffic ingestion.
- Integrate with a SIEM platform.
- Add alert prioritization and analyst feedback.
- Provide explainable-AI visualizations.
- Support concept-drift monitoring.
- Evaluate deep-learning and ensemble methods.
- Add role-based access and audit logging.

8. Model Development Roadmap

Planned Stage	What the Group Expects to Do	Expected Module 5 Output
Data preparation	Acquire the dataset, inspect files, clean values, encode categories, and create reproducible preprocessing pipelines	Prepared training, validation, and test data
EDA/feature engineering	Examine distributions, class balance, correlations, outliers, and feature importance; engineer only justified features	EDA report and feature-processing plan
Baseline	Train a majority-class classifier and a simple machine-learning model	Baseline metrics
Candidate model training	Train Random Forest and selected alternative models	Candidate model results
Evaluation	Compare models using macro F1, recall, precision, weighted F1, and confusion matrices	Model-comparison report
Error analysis	Investigate false positives, false negatives, and commonly confused attack categories	Error-analysis findings
Improvement/iteration	Tune hyperparameters, address class imbalance, and revise preprocessing where justified	Improved candidate model
Final model selection	Select the model that best balances performance, interpretability, reliability, and deployment feasibility	Final trained model
Application integration	Connect the saved model and preprocessing pipeline to a simple interface	Working AI application prototype

9. Proposed Application Concept

Application Element	High-Level Concept
Application type	Web-based demonstration dashboard or lightweight Streamlit application
Primary user	Security analyst or network administrator
User input	Network-traffic feature values or an uploaded record
Model output/result	Predicted class, confidence scores, and a brief interpretation
Key user interaction	Enter or upload traffic information, submit it for analysis, and review the result
Module 5 implementation focus	Build the interface, connect it to the trained model, validate inputs, and display results

The application will not automatically take defensive action. It will support a human analyst by organizing classification results in an accessible format.

10. Intended Deployment Direction

Deployment Element	High-Level Plan
Deployment environment	Local computer, classroom server, or lightweight cloud-hosted demonstration
User access	Browser-based access for authorized users
Model-serving direction	Load a serialized preprocessing pipeline and model through a Streamlit application or simple API
Key infrastructure/dependency	Python runtime, required libraries, model files, dataset or test input, and hosting environment
Main deployment risks	Environment incompatibility, unsupported input values, slow response, model-file errors, and security concerns if publicly exposed
Intended deployment level	Demonstration or limited pilot, not production-oriented deployment

Before any external deployment, access controls, secure configuration, logging, and protection of model files would be required.

11. Resources and Group Responsibilities

Area	Responsible Member(s)	Expected Contribution
Domain/problem research	Member 1	Research cybersecurity context, intrusion detection, and supporting evidence
Data discovery/preparation	Member 2	Acquire data, inspect quality, clean records, and document preprocessing
ML/DL development	Member 3	Train baseline and candidate models and perform evaluation
Application implementation	Member 4	Build the prototype interface and integrate the model
Deployment/testing	Members 3 and 4	Test the application, check latency, and prepare deployment files
Documentation/presentation	All members, coordinated by Member 1	Prepare proposal, technical report, demonstration, and presentation

11.1. Technical Resources

- NSL-KDD dataset.
- Python and Jupyter Notebook or Google Colab.
- Optional cloud hosting for Module 6 demonstration.

Before Module 5 begins, the group must confirm dataset access, repository access, development roles, and the chosen application framework.

12. Project Timeline

Stage	Key Activities	Expected Output	Responsible
Module 4: Proposal	Define the problem, users, data, ML task, MVP, evaluation criteria, risks, and roadmap	Approved project proposal	All members
Module 5: Data and baseline	Acquire data, clean records, conduct EDA, engineer features, and train baseline	Prepared data and baseline metrics	Data and ML members
Module 5: Model development	Train candidate models, tune parameters, evaluate performance, and analyze errors	Selected model candidate	ML members
Module 5: Implementation	Integrate the model into a prototype interface and perform functional testing	Working AI application	Application members
Module 6: Deployment	Configure serving environment and deploy the demonstration system	Deployed or deployment-ready prototype	Deployment members
Module 6: Validation	Test predictions, response time, reliability, and input handling	Validation results	All members
Final demonstration	Prepare presentation, documentation, limitations, and live demonstration	Capstone deliverables	All members

The group will reserve buffer time for dataset problems, model underperformance, application bugs, and deployment issues.

13. Risk Assessment and Mitigation

Risk	Likelihood	Impact	Mitigation/Fallback
Data availability or quality problems	Medium	High	Download and verify the dataset early; maintain a documented backup source such as KDD Cup 1999 or CIC-IDS2017
Class imbalance and minority-class performance	High	High	Use stratified splitting, macro metrics, class weights, and careful error analysis
Model performance below target	Medium	High	Compare multiple models, tune parameters, simplify label grouping if justified, and document limitations
Compute or resource limitations	Low to Medium	Medium	Use CPU-efficient models, sample data only when justified, and use cloud notebooks if needed
Implementation bugs	Medium	Medium	Use modular code, version control, input validation, and incremental testing
Deployment incompatibility	Medium	Medium	Pin package versions, create a requirements file, and test the application in a clean environment
Misinterpretation of predictions	Medium	High	Display confidence information, explain limitations, and require human review
Unauthorized exposure of the prototype	Low to Medium	High	Use authentication where possible, avoid real sensitive data, and restrict public access

14. Ethical, Privacy, and Responsible AI

Responsible AI Area	Issue/Risk Identified	Planned Safeguard
Privacy/data protection	Network data can potentially contain sensitive operational information	Use the public benchmark dataset; do not collect personal or confidential data
Bias/representativeness	Historical benchmark data may not represent modern or diverse network environments	State dataset limitations and avoid claiming universal performance
Fairness	Performance may differ substantially between attack classes	Report per-class metrics and investigate minority-class errors
Potential harm/misuse	Incorrect predictions could cause missed threats or unnecessary interventions	Use the model as decision support; do not automate blocking in the MVP
Transparency/explainability	Users may overtrust a prediction	Display the predicted class, confidence, evaluation summary, and limitations
Security/access control	A publicly exposed prototype could be misused	Restrict access, validate inputs, avoid sensitive data, and document secure-deployment requirements

The project will not claim that the model can detect every intrusion or replace a professional security operations team.

15. Expected Project Outcomes

Stage	Expected Outcome
Module 4	A feasible project proposal with a clearly defined cybersecurity problem, users, dataset plan, ML task, MVP, success criteria, risks, and roadmap
Module 5	A prepared dataset, trained and evaluated model, model-comparison results, and working application prototype
Module 6	A deployed or deployment-ready system that has been tested, validated, documented, and demonstrated
Practical value	A proof of concept showing how machine learning can help prioritize and classify network-security events

16. Future Improvements

Future Improvement	Limitation It Addresses	Why It Is Postponed
Evaluate a current intrusion-detection dataset	NSL-KDD may not reflect modern attack behavior	Requires additional data preparation and validation time
Add live traffic or streaming input	The MVP uses static records	Requires packet capture, streaming infrastructure, and stronger security controls
Integrate with SIEM or firewall systems	The MVP is a standalone prototype	Requires external system access, APIs, authentication, and operational testing
Add analyst feedback and retraining	The initial system uses a fixed trained model	Requires a feedback process and additional labeled data
Add explainable-AI tools	Basic feature importance may not explain individual predictions fully	Requires additional implementation and careful interpretation
Support concept-drift monitoring	Network behavior and attack patterns change over time	Requires long-term operational data and monitoring infrastructure

17. Executive Summary

Cyber attacks against computer networks can compromise confidential information, disrupt services, and create significant financial and operational losses. Network intrusion detection systems help organizations identify suspicious activity by analyzing network traffic. However, traditional approaches often depend on predefined rules and known attack signatures, which may fail to detect modified, novel, or complex attack patterns. Manual investigation can also be slow and difficult to scale as network traffic increases.

This project proposes a machine-learning system that classifies network traffic as either normal or one of several attack types. The project will use the NSL-KDD dataset, a publicly available benchmark dataset designed for network intrusion-detection research. The task is a supervised multi-class classification problem. Candidate models will include a baseline classifier, such as Logistic Regression or Decision Tree, and stronger models such as Random Forest, Gradient Boosting, or a neural network. The models will be evaluated using accuracy, precision, recall, F1-score, confusion matrices, and class-specific performance. Because failing to detect an attack can be more serious than producing a false alarm, macro-averaged recall and F1-score will receive particular attention.

The minimum viable product will be a working prototype that accepts network-traffic feature values and returns a predicted class, confidence information, and a basic explanation of the result. The prototype will be designed for demonstration or pilot use rather than production deployment. It will not attempt to monitor live enterprise traffic, automatically block attacks, or guarantee detection of previously unseen threats.

Module 4 will define the problem, dataset, machine-learning task, scope, evaluation criteria, and development plan. Module 5 will cover data preparation, exploratory analysis, model development, evaluation, and application implementation. Module 6 will focus on deployment, validation, performance checks, documentation, and final demonstration.

18. References

Use one consistent citation style throughout the final submission. The following sources are suitable starting points:

1. Tavallaee, M., Bagherzadeh, E., Lu, W., & Ghorbani, A. A. (2009). *A detailed analysis of the KDD CUP 99 data set*. Proceedings of the IEEE Symposium on Computational Intelligence for Security and Defense Applications.
2. Canadian Institute for Cybersecurity. *Intrusion detection datasets and cybersecurity research resources*. University of New Brunswick.
3. National Institute of Standards and Technology. *Computer Security Incident Handling Guide*. NIST Special Publication 800-61.
4. Scikit-learn documentation. *Classification, preprocessing, ensemble methods, and model evaluation*. https://scikit-learn.org/
5. NSL-KDD Dataset. Public benchmark dataset for network intrusion-detection research. Cite the repository actually used by the group, such as the verified Kaggle or academic-hosted source.